

CM11 : Réseaux de neurones

1 Introduction

Rappelons l'équation d'un hyper-plan :

$$\mathcal{H} = \{\vec{x} \mid \vec{w} \cdot \vec{x} + b = 0\}$$

On peut écrire la fonction de prédiction pour tout $\vec{x}$:

$$h(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

L'algorithme du perceptron consiste à trouver $\vec{w}$ et b tels que l'hyperplan associé sépare au mieux nos données.

Quelle est la limite de cette méthode ?

Quelle solution avons nous vu ?

En notant ϕ la fonction de projection de nos données, on obtient pour fonction de prédiction :

$$h(\vec{x}) = \vec{w} \cdot \phi(\vec{x}) + b$$

L'idée des réseaux de neurones est d'**apprendre cette fonction de projection** à partir des données. La fonction projection s'écrit :

$$\phi(\vec{x}) = \sigma(U\vec{x} + \vec{c})$$

Où σ est la **fonction d'activation**, plusieurs sont possibles (section suivante), c'est un choix d'architecture, l'important est que cette fonction soit **non-linéaire**. U est une matrice de taille (H, D) qui est un **paramètre** du modèle, elle va être modifiée au cours de l'apprentissage. H , la première dimension de la matrice est un **hyperparamètre** du modèle, c'est la largeur du réseau, plus H est grand, plus le modèle est complexe. Le vecteur $\vec{c}$ est de taille H , il correspond au biais.

La fonction ϕ est une projection de quel espace dans quel espace ?

2 Fonction d'activation

Supposons l'absence de la fonction d'activation, alors le classifieur serait linéaire, il n'y aurait aucun gain par rapport au perceptron. En effet, en supposant l'absence de la fonction d'activation σ :

$$h(\vec{x}) = \vec{w}^\top \phi(\vec{x}) + b = \vec{w}^\top (U\vec{x} + \vec{c}) + b = \vec{w}^\top U\vec{x} + \vec{w}^\top \vec{c} + b = \vec{w}'^\top \vec{x} + b'$$

En notant $\vec{w}' = \vec{w}^\top U$, qui est bien un vecteur de $\mathbb{R}^D$ et $b' = \vec{w} \cdot \vec{c} + b$ qui est bien un scalaire. C'est d'ailleurs un bon exercice pour vous que de vérifier la cohérence des dimensions de chaque vecteur et matrice.

Les fonctions d'activations populaires sont par exemple la sigmoïde, tanh, ReLU ou encore leaky ReLU. Sigmoïde et tanh sont des historiquement populaires, actuellement ReLU ou leaky ReLU sont les plus utilisées.

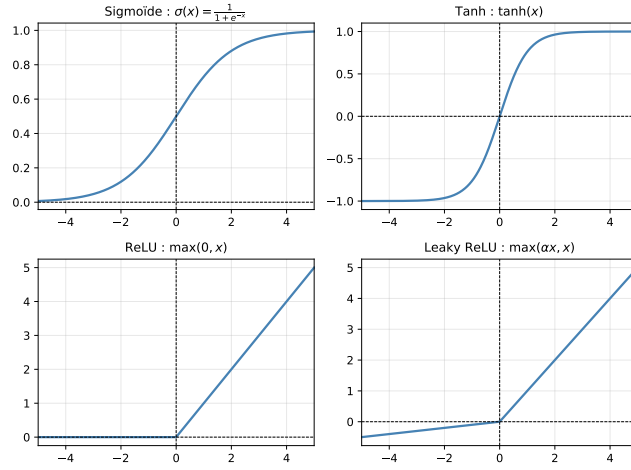


Figure 1: Fonctions d'activations.

3 Théorème d'approximation universelle

Lors du cours sur les features maps, nous avons notamment vu les features maps polynomiales. Par exemple, dans le cas de données de dimension 3, la feature map polynomiale d'ordre 2 s'écrivait :

$$\vec{\phi}(\vec{x}) = (1, x_1, x_2, x_3, x_1x_2, x_1x_3, x_2x_3, x_1^2, x_2^2, x_3^2)$$

On peut se demander si la projection apprise par un réseau de neurone, c'est à dire de la forme $\phi(\vec{x}) = \sigma(U\vec{x} + \vec{c})$, peut être équivalente à cette feature map polynomiale. La réponse est oui, c'est le théorème d'approximation universelle. Ce théorème indique qu'un réseau de neurone d'une seule couche cachée (c'est à dire ce que nous venons de définir) peut approximer toute fonction continue. L'approximation étant meilleure plus le paramètre H (la largeur du réseau) est grand.

Ce résultat peut sembler surprenant : comment une simple composition $\sigma(U\vec{x} + \vec{c})$ peut-elle approximer des fonctions aussi complexes qu'une feature

map polynomiale ? Nous n'allons pas donner la preuve du théorème, mais l'intuition.

Pour cela nous allons étudier un exemple en dimension 1, c'est à dire $\vec{x} = x$ un scalaire. Choisissons pour σ la fonction ReLU. Nous avons alors :

$$h(\vec{x}) = \vec{w} \cdot \phi(\vec{x}) + b \quad (1)$$

$$= \vec{w} \cdot \sigma(U\vec{x} + \vec{c}) + b \quad (2)$$

$$= \vec{w} \cdot \sigma \left(\begin{pmatrix} u_1 x + c_1 \\ \vdots \\ u_H x + c_H \end{pmatrix} \right) + b \quad (3)$$

$$= \vec{w} \cdot \begin{pmatrix} \sigma(u_1 x + c_1) \\ \vdots \\ \sigma(u_H x + c_H) \end{pmatrix} + b \quad (4)$$

$$= b + \sum_h w_h \sigma(u_h x + c_h) \quad (5)$$

$$= b + \sum_h w_h \max(u_h x + c_h, 0) \quad (6)$$

Avec (6) la définition de ReLU.

Nous voyons donc que le résultat final est une somme de fonctions ReLU avec différents paramètres. On visualise le résultat avec la figure 2.

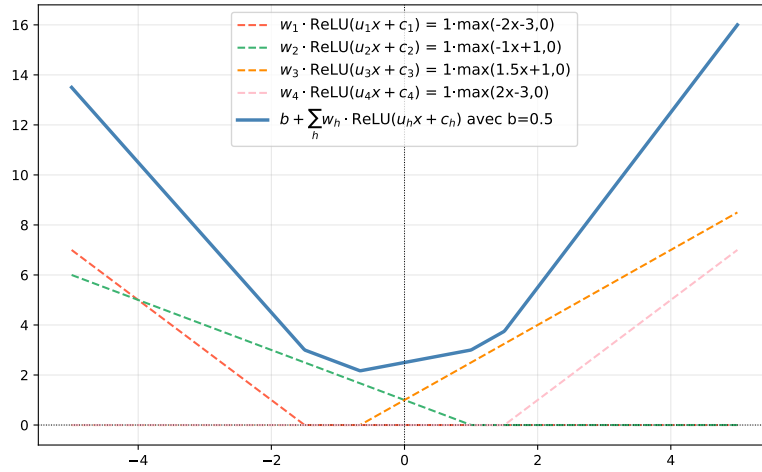


Figure 2: Illustration en dimension 1 de la fonction $h(\vec{x}) = \vec{w} \cdot \sigma(U\vec{x} + \vec{c}) + b$. Ici $H = 4$ et les paramètres U , $\vec{c}$, $\vec{w}$ et b sont fixés arbitrairement. Le résultat final est la somme de 4 fonctions ReLU.

4 Optimisation des paramètres

Pour l'algorithme du perceptron nous devons optimiser un paramètre, $\vec{w}$. Oubliions le biais pour le moment (on peut l'inclure dans les données en ajoutant une dimension), on veut minimiser la loss :

$$\mathcal{L}(\Theta, X, Y) = \frac{1}{N} \sum_{n=1}^N l(\Theta, x_n, y_n) = \frac{1}{N} \sum_{n=1}^N \log(1 + e^{-y_n h(\vec{x}_n)})$$

Cette loss est nouvelle, ce n'est pas celle que nous avons vu pour le perceptron, mais la log loss qui est davantage adaptée pour le MLP (et le deeplearning en général).

Comment minimiser la loss ? Descente de gradient.

$$\frac{\partial l}{\partial h} = \frac{-y_n e^{-y_n h}}{1 + e^{-y_n h}} \quad (7)$$

$$= \frac{-y_n}{1 + e^{y_n h}} \quad (8)$$

$$\frac{\partial l}{\partial b} = \frac{\partial l}{\partial h} \cdot \frac{\partial h}{\partial b} \quad (9)$$

$$= \frac{\partial l}{\partial h} \cdot \frac{\partial(\vec{w} \cdot \phi(\vec{x}_n) + b)}{\partial b} \quad (10)$$

$$= \frac{\partial l}{\partial h} \quad (11)$$

$$\frac{\partial l}{\partial \vec{w}} = \frac{\partial l}{\partial h} \cdot \frac{\partial h}{\partial \vec{w}} \quad (12)$$

$$= \frac{\partial l}{\partial h} \cdot \frac{\partial(\vec{w} \cdot \phi(\vec{x}_n) + b)}{\partial \vec{w}} \quad (13)$$

$$= \frac{\partial l}{\partial h} \cdot \phi(\vec{x}_n) \quad (14)$$

et

$$\frac{\partial l}{\partial U} = \frac{\partial l}{\partial h} \cdot \frac{\partial h}{\partial U} \quad (15)$$

$$= \frac{\partial l}{\partial h} \cdot \frac{\partial(\vec{w} \cdot \sigma(U \vec{x}_n) + b)}{\partial U} \quad (16)$$

$$= \frac{\partial l}{\partial h} \cdot \vec{w} \cdot \frac{\partial \sigma(U \vec{x}_n)}{\partial U} \quad (17)$$

$$= \frac{\partial l}{\partial h} \cdot \vec{w} \cdot \sigma'(U \vec{x}_n) \vec{x}_n \quad (18)$$

5 Réseaux de neurones multi-couches

Nous avons vu que nous pouvions apprendre la projection de nos données avec la fonction de projection :

$$\phi(\vec{x}) = \sigma(U\vec{x} + \vec{c})$$

La fonction de prédiction étant ensuite la même que pour le perceptron :

$$h(\vec{x}) = \vec{w} \cdot \phi(\vec{x}) + b$$

Cela correspond à un réseau de neurone à une couche cachée et comme nous l'avons vu, avec une largeur suffisante, cela permet déjà d'approximer n'importe quelle fonction continue. On pourrait donc penser que cela est suffisant, mais cela a plusieurs désavantages :

- La largeur du modèle (H , le nombre de lignes de la matrice U) nécessaire à une bonne approximation peut être bien trop important.
- Un réseau un réseau profond peut réutiliser des représentations intermédiaires, ce qui permet d'atteindre la même capacité d'approximation avec beaucoup moins de neurones et limite le risque d'overfit.

Une méthode pour contrer ces deux problèmes est d'ajouter des couches à notre réseau de la manière suivante :

$$\phi(\vec{x}) = \sigma(U\phi'(\vec{x}) + \vec{c})$$

avec

$$\phi'(\vec{x}) = \sigma(U'\vec{x} + \vec{c}')$$

On emboîte ainsi les fonctions de projection, ce qui permet de construire progressivement des représentations plus riches des données. La profondeur du réseau est alors le nombre de couches ainsi définies. Si l'on garde seulement ϕ et ϕ' comme ci dessus, notre réseau à deux couches cachées.

5.1 Descente de gradient, intuition

L'optimisation des paramètres se fait par descente de gradient. Cela pourrait sembler un calcul extrêmement compliqué à mesure que l'on ajoute des couches, mais en fait non, grâce au théorème de dérivation des fonctions composées. Pour saisir l'idée, prenons l'exemple de trois couches cachées, c'est à dire :

$$\begin{aligned}\phi(\vec{x}) &= \sigma(U\phi'(\vec{x}) + \vec{c}) \\ \phi'(\vec{x}) &= \sigma(U'\phi''(\vec{x}) + \vec{c}') \\ \phi''(\vec{x}) &= \sigma(U''\vec{x} + \vec{c}'')\end{aligned}$$

et notons :

$$\begin{aligned}a''(\vec{x}) &= U''\vec{x} + \vec{c}'' \\a'(\vec{x}) &= U'\phi''(\vec{x}) + \vec{c}' = U'\sigma(a''(\vec{x})) + \vec{c}' \\a(\vec{x}) &= U\phi'(\vec{x}) + \vec{c} = U\sigma(a'(\vec{x})) + \vec{c}\end{aligned}$$

On a alors,

$$\begin{aligned}\frac{\partial l}{\partial U} &= \frac{\partial l}{\partial a} \cdot \frac{\partial a}{\partial U} \\ \frac{\partial l}{\partial U'} &= \frac{\partial l}{\partial a} \cdot \frac{\partial a}{\partial a'} \cdot \frac{\partial a'}{\partial U'} \\ \frac{\partial l}{\partial U''} &= \frac{\partial l}{\partial a} \cdot \frac{\partial a}{\partial a'} \cdot \frac{\partial a'}{\partial a''} \cdot \frac{\partial a''}{\partial U''}\end{aligned}$$

On peut voir que pour chaque couche cachée il ne faut pas recalculer le gradient "depuis le début", il y a seulement deux nouveaux termes à calculer et l'on connaît en fait déjà un.

Exercice : calculez ces deux termes.

6 Stratégies d'optimisation

Pour l'algorithme du perceptron nous avons vu deux manières d'effectuer la descente de gradient, soit en mettant à jour les paramètres après chaque point de donnée, un par un (implémentation donnée dans le CM3), soit en calculant le gradient sur l'ensemble des données à chaque itération (implémentation que nous avons vu en TP).

Dans le cas du perceptron, cela ne change pas grand chose, si les données sont linéairement séparables, les deux méthodes de descente de gradient convergent. De plus pour le perceptron seul les problèmes liés au temps de calcul sont plutôt absents.

En général on distingue trois stratégies de descente de gradient :

- **Gradient descent (batch)** : on calcule le gradient sur l'ensemble des données à chaque itération. C'est stable mais coûteux si N est grand.
- **Stochastic gradient descent (SGD)** : on met à jour les paramètres après chaque point, un par un. Le gradient est alors une estimation bruitée du gradient global, mais il reste correct en moyenne. Ce bruit peut se révéler un avantage en évitant de tomber dans des minima locaux.
- **Mini-batch gradient descent** : un compromis, on divise les données en petits groupes (mini-batches) et on calcule le gradient sur chaque groupe. **C'est ce qu'on utilise en pratique en deep learning.** Le nombre de données par mini-batch est un choix, il est généralement entre 16

et quelques centaines, mais parfois beaucoup plus dans les modèles très larges, il n'y a pas une règle absolue. L'avantage de mini-batch par rapport à SGD est de pouvoir paralléliser les calculs.